

Cairn v0.1.1 Launch Failure – Detailed Engineering PostMortem

Executive Summary

The application worked correctly during development (pnpm dev) but failed after being packaged and installed through the Windows installer.

Users encountered the error 'Can't find meta/_journal.json file' during onboarding. The issue was traced to a migration path mismatch in the packaged Electron application.

Symptoms Observed

- Application launched successfully.
- UI rendered correctly.
- Database initialization began.
- Drizzle migration engine attempted to execute migrations.
- Application failed while searching for meta/_journal.json.
- Onboarding became unusable.

Initial Hypotheses

1. Migration files were not included in the installer.
2. _journal.json was corrupted.
3. Runtime migration path was incorrect after packaging.

Investigation Step 1 – Verify File Exists

We inspected electron/db/migrations/meta/_journal.json.

The file existed and contained valid JSON describing all migration entries.

Conclusion: file corruption was ruled out.

Investigation Step 2 – Verify Packaging

Command executed:

```
dir /s "_journal.json"
```

Result confirmed the file existed inside the project source tree.

Investigation Step 3 – Inspect app.asar

Command executed:

```
npm asar list dist\win-unpacked\resources\app.asar | findstr migrations
```

Result showed:

```
electron\db\migrations\0001_initial.sql
```

```
...
```

```
electron\db\migrations\meta\_journal.json
```

Conclusion:

The migration files WERE packaged into app.asar.

Root Cause Discovery

The code responsible for locating migrations was:

```
const migrationsFolder = app.isPackaged
  ? join(process.resourcesPath, 'migrations')
  : join(__dirname, '..', '..', 'electron', 'db', 'migrations')
```

During development this worked.

After packaging, Electron searched:
resources/migrations

However the actual files were stored inside:
resources/app.asar/electron/db/migrations

Therefore Drizzle searched the wrong directory and could not locate meta/_journal.json.

Packaging Configuration Review

Original electron-builder.yml:

```
files:
- out/**/*
- package.json
```

This did not explicitly include the migration folder.
Although the files ended up inside app.asar, packaging behavior was not guaranteed.

Packaging Fix Applied

Updated electron-builder.yml:

```
files:
- out/**/*
- electron/db/**/*
- package.json
- "!(**/.{git,svn})"
- "!(**/node_modules/.cache"
```

This guarantees migration assets are packaged.

Runtime Code Fix Applied

BEFORE:

```
const migrationsFolder = app.isPackaged
  ? join(process.resourcesPath, 'migrations')
  : join(__dirname, '..', '..', 'electron', 'db', 'migrations')
```

AFTER:

```
const migrationsFolder = app.isPackaged
  ? join(process.resourcesPath, 'app.asar', 'electron', 'db', 'migrations')
  : join(__dirname, '..', '..', 'electron', 'db', 'migrations')
```

The packaged build now points to the actual migration location.

Verification

After rebuilding:

```
pnpm dist
```

We re-ran:

```
npx asar list dist\win-unpacked\resources\app.asar | findstr migrations
```

The output confirmed all SQL migrations and meta/_journal.json were present.

The installer was rebuilt as:
Cairn Setup 0.1.1.exe

Installation and onboarding completed successfully.

Timeline

1. Error reproduced in packaged installer.
2. Verified JSON file integrity.
3. Verified file existence.
4. Inspected packaged app.asar.
5. Confirmed migration assets were packaged.
6. Traced runtime path resolution.
7. Identified path mismatch.
8. Updated electron-builder.yml.
9. Updated migration folder resolution logic.
10. Rebuilt installer.
11. Verified successful onboarding.

Lessons Learned

- Always verify packaged paths separately from development paths.
- Electron app.asar changes runtime file locations.
- Migration assets should be explicitly packaged.
- Add installer testing as part of release validation.
- Validate all database initialization paths before publishing releases.

Preventive Actions

1. Add startup logging for migration paths.
2. Add packaged-build smoke tests.
3. Add installer validation checklist.
4. Verify migration folder existence before running migrate().
5. Consider automated CI packaging tests.

Final Root Cause Statement

The application failed because Drizzle ORM migrations were packaged inside:

```
resources/app.asar/electron/db/migrations
```

but the runtime code attempted to load them from:

```
resources/migrations
```

This path mismatch prevented Drizzle from finding meta/_journal.json, causing database initialization to fail during onboarding. Updating the runtime migration path and explicitly packaging

migration assets fully resolved the issue in Cairn v0.1.1.